Tools — Utility

Introspection of the running ragent binary, the active LLM connection, and the host environment.

Tool	Description
`model_info`	Report the active provider/model, capabilities, context window, and cost tier.
`ragent_info`	Report the running ragent version, build time, git commit, compiler, and execution details (pid, uptime, working directory, user, memory, threads).
`os_info`	Report read-only host OS and hardware introspection (also `/osinfo show`).
`tool_info`	Dump the whole tool registry as JSON (name, description, schema, permission category, source).
`commands_info`	Dump the slash-command catalog as JSON (built-in TUI commands plus plugin-contributed ones).
`DateTime`	Current date and time from the system clock: RFC 3339 UTC instant, an optional fixed-offset rendering, the individual fields, and the Unix timestamp.

---

model_info

Report the active provider/model pair, provider display name, capabilities, context window, max output tokens, cost tier, and thinking support. When the Model Router is active, also reports whether routing is enabled and that the effective downstream model is chosen per request.

Arguments

Argument	Type	Required	Description	Typical value
`format`	enum	no	`text` (human-readable markdown, default) or `json` (structured metadata only)	`"text"`

Example:

```
model_info
model_info format="json"
```

---

ragent_info

Report build and version information about the running ragent binary - the ragent version, the build timestamp, the git commit it was built from (best-effort), and the compiler version - together with runtime

execution details of the current instance: process id and parent process id, wall-clock start time, uptime, absolute executable path, current working directory, user (from `$USER` / `$USERNAME` / `$LOGNAME`), resident and virtual memory, and thread (task) count. Read-only and offline - it never shells out or hits the network (only the current process is inspected via `sysinfo`) - so the LLM can answer "which version am I running, and how long has this instance been up?" directly. Every execution field is best-effort and renders as `unknown` when the host does not expose it.

Arguments

Argument	Type	Required	Description	Typical value
`format`	enum	no	`text` (human-readable markdown, default) or `json` (structured metadata only)	`"text"`

Example:

```
ragent_info
ragent_info format="json"
```

---

os_info

Report read-only information about the host operating system and hardware: OS identity and Linux distribution, CPU, graphics adapters and graphics-API versions, physical hardware (system/chassis/motherboard/BIOS identity, storage devices, network interfaces), memory, uptime, and the ragent process environment. It always succeeds, reporting `unknown`/`0` placeholders for any value the host cannot supply, and never reads serial numbers, UUIDs, or asset tags. Also available as the `/osinfo show` slash command.

By default (`probe = true`) it runs a fixed, timeout-bounded allowlist of vendor graphics diagnostics (`vulkaninfo`, `glxinfo`, `nvidia-smi`, `rocminfo`, `system_profiler`) so every graphics API is reported with its version - OpenGL, OpenGL ES, and Mesa expose no read-only version file. Those diagnostics only read host state; set `probe: false` for a fully process-free pass.

Arguments

Argument	Type	Required	Description	Typical value
`format`	enum	no	`text` (human-readable markdown, default) or `json` (structured metadata only)	`"text"`

Argument	Type	Required	Description	Typical value
`probe`	bool	no	Run the allowlisted graphics diagnostics (default `true`); `false` for a process-free report	`false`

Example:

```
os_info
os_info format="json" probe=false
```

---

tool_info

Return a JSON-encoded dump of the tool registry: every registered tool with its name, description, parameters JSON schema, permission category, source (`internal` / `mcp:<server>` / `plugin:<id>` / `visibility:<family>`), and hidden state, plus MCP server/tool provenance for bridged tools. Read-only, permission category `none`, hardwired auto-approved.

Arguments

None.

Example:

```
tool_info
```

---

commands_info

Return a JSON-encoded catalog of every slash command — the built-in TUI commands (trigger, description, subcommands, flags) plus the commands contributed by enabled plugins, resolved live at call time. Read-only, permission category `none`, hardwired auto-approved.

Arguments

None.

Example:

```
commands_info
```

DateTime

Return the current date and time read from the system clock. Reports the UTC instant in RFC 3339, the same instant rendered at an optional fixed UTC offset, the individual date and time fields, the day of week, and the Unix timestamp in both seconds and milliseconds. Read-only, permission category `none`.

The tool reads neither the `TZ` environment variable nor the host timezone database, so a result never depends on ambient process state and the same call answers the same way on every machine; the offset it used is echoed in the metadata. The TUI renders the call as a `UTC+HH:MM` zone summary with the formatted instant as the result.

Arguments

Argument	Type	Required	Description	Typical value
`timezone_offset_minutes`	integer	no	Fixed UTC offset for the rendered local form, between -1080 and 1080 (0 is UTC, 60 is UTC+01:00, -300 is UTC-05:00, 330 is UTC+05:30). Defaults to 0.	`60`
`format`	string	no	chrono date-time format string applied to the local rendering	`"%d/%m/%Y %H:%M"`

Example:

```
DateTime
DateTime timezone_offset_minutes=60
DateTime timezone_offset_minutes=-300 format="%d/%m/%Y %H:%M"
```